CMPUT 398 Optional Project

GUI Next Action Planner

Zongxin Liu

1762546

zongxin

1. Introduction

With rapid advances in artificial intelligence, Vision-Language Models (VLMs) combine computer vision and natural language processing, allowing systems to understand both images and text together (Alain Airom (Ayrom 2025). At the same time, modern user interfaces are becoming increasingly complex, with dense screens, nested menus, and app-specific interaction patterns. These Graphical User Interfaces (GUIs) are the primary way people interact with modern software. While GUIs are designed to be intuitive for humans, they are difficult for machines to interact with because layouts can change across devices, and actions often depend on visual understanding rather than fixed commands. Along with the lack of available APIs to the application, this poses a challenge for automating tasks. This challenge has led to growing interest in GUI agents, systems that can understand user goals, interpret interface elements, and perform actions within graphical interfaces on behalf of users ("How Smart Is Your GUI Agent? A Framework for the Future of Software Interaction," 2018).

This project focuses on "next action" planning from a smartphone screenshot and a user-provided goal, such as opening the YouTube app. The motivation of the project is to explore the feasibility of building a GUI agent using current vision-language models (VLMs), investigating whether these models are capable of understanding a graphical interface and recommending actions that accomplish a user's goal. To achieve this, the project adopts a client–server architecture in which a React frontend allows users to upload a screenshot and enter a goal, while a Flask backend sends this information to a vision-language model through an external API and returns a recommended next action. This design allows lightweight devices to access advanced big reasoning models running in a remote server, typically operated by the model provider.

A possible alternative would be to run the vision-language model directly on the user's device. This would eliminate the need for an external API and the possible cost for each API call. However, most modern multimodal models require large amounts of memory and computing resources, making them difficult to run efficiently on standard laptops or mobile devices. For this reason, this project adopts a client–server architecture in which the frontend communicates with a Flask backend server, which in turn interacts with an external vision-language model API to perform the computationally intensive inference.

2. Design, Implementation

2.a. System Design

The frontend is implemented in React and provides a workflow for uploading a screenshot, entering a goal, and choosing a VLM. It exposes three main views: a Planner page for creating plans, a Plans list for browsing past plans, and a Plan detail page for viewing and deleting individual plans. The backend is implemented in Flask and uses an SQLite database. As shown in Figure 1, a table named "plans" is defined to store plan records. Each record stores the goal, the selected model, the VLM-generated next-action recommendation, the screenshot (as base64), the MIME type, and a timestamp.

```
class Plan(db.Model):
    __tablename__ = "plans"

    id = db.Column(db.Integer, primary_key=True, autoincrement=True)
    goal = db.Column(db.Text, nullable=False)
    model = db.Column(db.Text, nullable=False)
    action = db.Column(db.Text, nullable=False)
    screenshot_base64 = db.Column(db.Text, nullable=False)
    mime_type = db.Column(db.Text, nullable=False)
    created_at = db.Column(db.DateTime, nullable=False, default=lambda: datetime.now(timezone.utc))
```

Figure 1. Code Snippet of defining the database schema

The backend server interacts with the database through a REST API. It exposes endpoints for creating plans (POST /plans), listing all plans (GET /plans), retrieving a single plan by ID (GET /plans/:id), and deleting a plan (DELETE /plans/:id), as shown in Table 1.

Method	Endpoint	Usage	Request
GET	/plans	Get all the plans stored in the database	
POST	/plans	Create a new plan, call the external VLM API to generate an action, and store it in the database	- image: screenshot file (required) - goal: user goal (required) - model: model name (optional, default to glm-4.6v-flash
GET	/plans/<int:plan_id>	Retrieve a specific plan based on plan_id	
DELETE	/plans/<int:plan_id>	Delete a specific plan based on plan_id	

Table 1. REST API endpoints of the system

When a user submits a screenshot and goal, the server forwards the received image and instruction to a selected VLM (e.g. GLM-4.6V-Flash) via an external API using a structured prompt, and stores the returned action together with the screenshot and metadata in the database. The frontend can then display the recommended action and allow the user to browse, view, and delete plans over time.

2.b. REST:

REST architecture is chosen as this system is simple and request-based. To illustrate, the client sends a request containing a screenshot and a goal, the server processes the request using a VLM API, and the server returns the result. Based on Table 1, the server can also send requests to retrieve either a single record or all records from the database, as well as delete a specific record. It is clear that one request corresponds to one response, and the backend server does not need to remember the state of the client. REST works well for this pattern because it is stateless, meaning that each request is independent and contains everything the server needs. It is also simple to implement using standard HTTP methods and widely supported by browsers and web frameworks such as React and Flask.

WebSockets were not chosen for this system because they are stateful and require maintaining persistent connections between the client and server. They are typically designed for applications that require streaming data or live bidirectional messaging. Since this system processes each request independently, WebSocket is not the best fit. Another architectural style is gRPC, which also uses persistent connections and streams that allow multiple messages. It's better suited for internal systems that require real-time streaming and large data loads (Amazon Web Services 2026). In the context of this project, gRPC would introduce additional complexity compared to a simple HTTP-based REST API.

2.c. Prompt Engineering and VLM API

To generate the next UI action, the backend server communicates with a vision-language model through an external API. The screenshot is first encoded as a Base64 image as shown in Figure 2, and later included in the request together with the user's goal since Image URL or Base64

encoding are two acceptable formats for sending an image to GLM-4.6V-flash model, according to the API documentation.

```python
def image_file_to_base64(image_file) -> str:
    """Convert an uploaded image file into a base64 string."""
    image_bytes = image_file.read()
    image_file.seek(0)
    return base64.b64encode(image_bytes).decode("utf-8")
```

Figure 2. Code Snippet for Image encoding

```python
system_prompt = (
    "You are a mobile agent that recommends the next UI action from a phone screenshot and a user goal."
)
user_prompt = f"Goal: {goal}\nReturn the next UI action only."
data_url = f"data:{mime_type}; base64,{image_b64}"
```

Figure 3. System prompt and user prompt

The API call follows a chat-based message format that includes both a system prompt and a user prompt. The system prompt defines the role and behaviour of the model, instructing it to act as a mobile agent, as shown in Figure 3. This gives the model context and helps guide the model toward producing consistent and task-specific outputs. The user prompt contains the actual task context and is more specific, including the user's goal, such as "open YouTube app," and instructs the model to return only the recommended next action. Separating the prompts in this way is a common prompt engineering practice that improves reliability by clearly distinguishing the general instructions for the model from the specific user request.

Following the API documentation, the combined prompts and screenshot are sent to the VLM through an API request, and the server extracts the generated action from the response before storing it in the database and returning it to the client, as shown in Figure 4.

```python
    url = "https://api.z.ai/api/paas/v4/chat/completions/"
    messages = [
        {"role": "system", "content": system_prompt},
        {
            "role": "user",
            "content": [
                {"type": "text", "text": user_prompt},
                {"type": "image_url", "image_url": {"url": data_url}},
            ],
        },
    ]
    payload = {"model": "glm-4.6v-flash", "messages": messages}
    headers = {"Authorization": f"Bearer {api_key}", "Content-Type": "application/json"}
    resp = requests.post(url, headers=headers, json=payload, timeout=60)
    if resp.ok:
        data = resp.json()
        return data["choices"][0]["message"]["content"]
    return f"[Error] GLM request failed ({resp.status_code}): {resp.text[:500]}"
```

Figure 4. Code snippet of an external API call for the VLM

3. Empirical Evaluation

This section presents examples demonstrating the functionality of the system using direct HTTP requests through curl commands as well as through the React frontend. The goal is to evaluate the correctness of the API and inspect the model performance.

3.a. Example 1:

```
> curl -X POST http://127.0.0.1:5001/plans \
  -F "image=@/Users/galvinliu/Downloads/Agoda_homescreen.png" \
  -F "goal=Find a hotels near New York" \
  -F "model=glm-4.6v-flash"
```

```
{
  "action": "\nTap on the \"Hotels\" section at the top of the screen.",
  "created_at": "2026-03-09T21:42:58.832813",
  "goal": "Find a hotels near New York",
  "id": 7,
  "mime_type": "image/png",
  "model": "glm-4.6v-flash"
}
```

Figure 5. Curl POST Request and Response to “/plans” endpoint

Inspecting Figure 5, it is clear that the curl request to the backend server running on port 5001 is successful. A JSON response with the action returned from the VLM and metadata is returned.

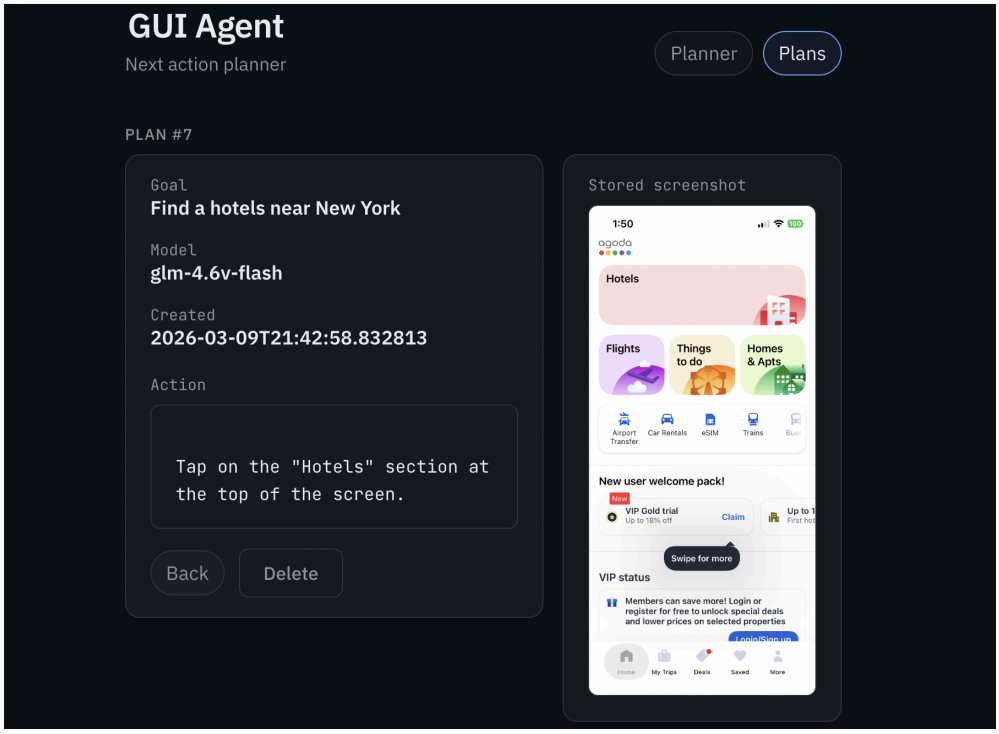

Figure 6. Front-end Plan Details page for Plan 7

Meanwhile, the frontend also triggers the "/plans/7" endpoint to retrieve the details of the newly created plan. As shown in the screenshot above, the model returned a reasonable action by recommending that the user tap the "Hotels" section at the top of the screen. It identifies both the button to click and its location on the screen, indicating that the VLM can interpret the visual components on the current screenshot, understand the spatial layout of interface elements, and generate an appropriate action based on the user's goal.

3.b. Example 2:

```
curl -X POST http://127.0.0.1:5001/plans \
  -F "image=@/Users/galvinliu/Downloads/Youtube_homescreen.png" \
  -F "goal=Change the language from English to french" \
  -F "model=glm-4.6v-flash"
```

```
{
  "action": "\nTap on the \"You\" icon (green circle with 'G') at the bottom right corner of the scr
een.",
  "created_at": "2026-03-09T22:49:17.340249",
  "goal": "Change the language from English to french",
  "id": 8,
  "mime_type": "image/png",
  "model": "glm-4.6v-flash"
}
```


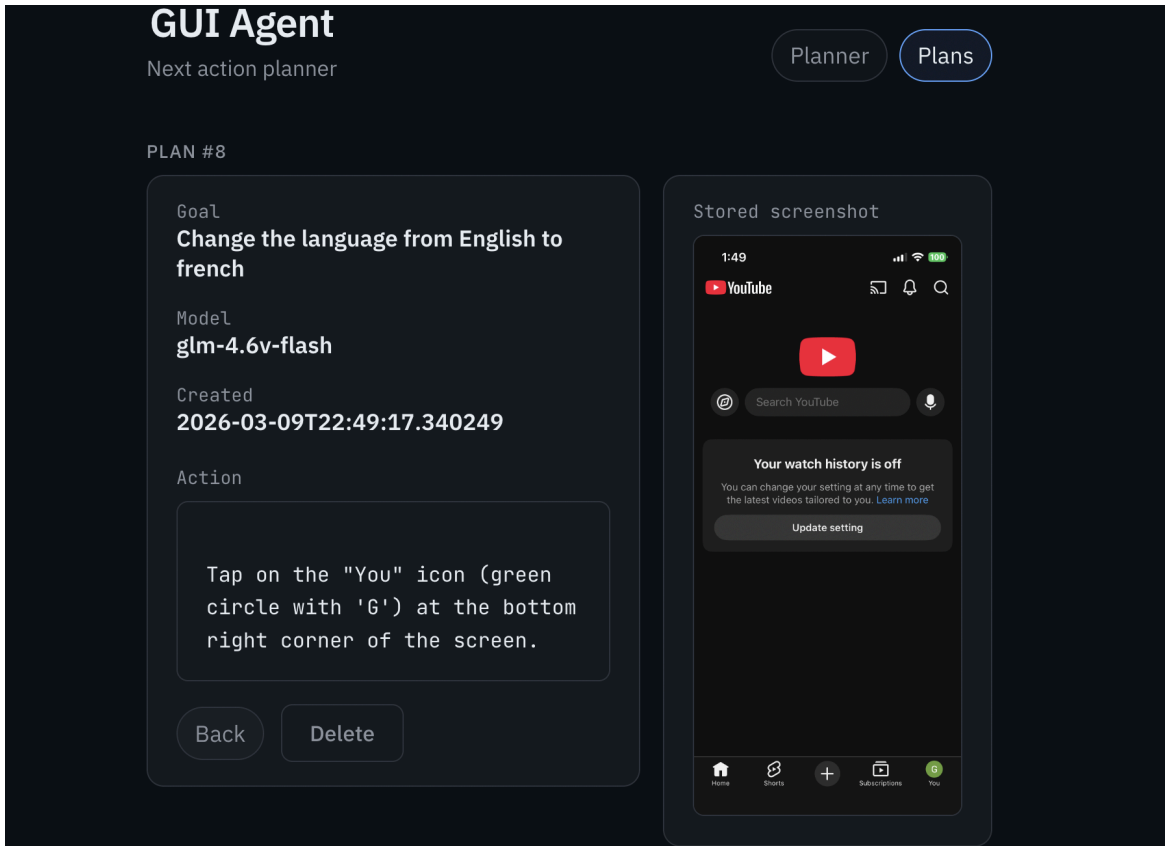

Figure 7. Curl Request, response, and Front-end Plan details page

As shown in Figure 7, the curl request is successful and returns a reasonable next step. The goal is to change the language from English to French. For a human user, this process typically involves navigating through several layers of the interface: first opening the profile page (the "You" button at the bottom-right corner), then entering Settings, and finally selecting Languages to change the application language. Although the provided screenshot only shows the YouTube home screen, where there is no visible language option, the VLM correctly recommends tapping

the “You” icon, which is the first step required to reach the settings menu. This suggests that the model is not only interpreting the visual elements on the screen but is also able to reason about the interface hierarchy and the sequence of actions needed to achieve the user’s goal.

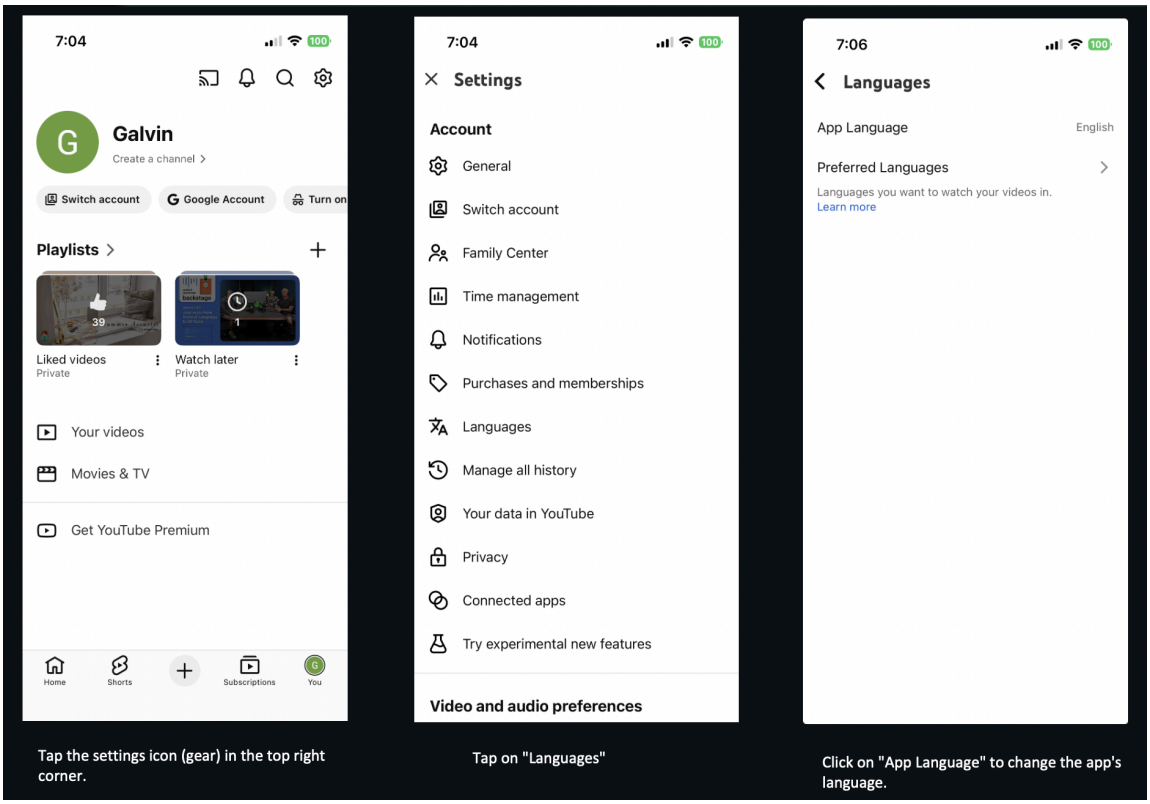

Figure 8. A sequence of plans with an action returned by the system

The same goal, “Change the language from English to French,” is used with a sequence of screenshots taken after performing each previously recommended action. As shown in the three screenshots above, each new screenshot represents the interface after completing the prior step. For each stage, the next-action planner produces the same navigation steps that a human user would normally follow to change the language. This behaviour suggests that the VLM is capable of goal-oriented planning, rather than simply identifying visible interface elements on the current screen.

4. Concluding Remark

The advancement of VLMs creates a way for systems to understand both visual content and natural language, along with the increasing need for GUI agents to perform tasks on many applications. This project explores the feasibility of using vision-language models to support GUI agents that recommend actions based on screenshots and user goals. By implementing a client–server architecture with a React frontend, Flask backend, and an external VLM API, the system demonstrates how multimodal models can interpret interface elements and generate reasonable next-step actions. While the system currently focuses on recommending actions rather than executing them, the results suggest that VLMs provide a promising foundation for future GUI agents that assist users in navigating complex applications or performing repetitive tasks.

References:

Alain Airom (Ayrom. 2025. "What Are Vision-Language Models (VLMs) and How Do They Work?" Medium. June 17, 2025. https://alain-airom.medium.com/what-are-vision-language-models-vlms-and-how-do-they-work-592ad38e0d3e.

"How Smart Is Your GUI Agent? A Framework for the Future of Software Interaction." 2018. Arxiv.org. 2018. https://arxiv.org/html/2602.11514.

Amazon Web Services. 2026. "What's the Difference between GRPC and REST?" Amazon Web Services, Inc. February 12, 2026. https://aws.amazon.com/compare/the-difference-between-grpc-and-rest/.